LABORATORIO 2

DISEÑO DE CÓDIGO – PROMEDIO DE ESTUDIANTES

MELISA GUZMÁN MUÑOZ

DISEÑO DE DATOS Y ALGORITMOS

EDWARD ALEXANDER FRANCIA GUZMAN

ESCUELA COLOMBIANA DE INGENIERÍA JULIO GARAVITO

BOGOTÁ, 2026

Descripción del problema:

Se necesita calcular el promedio de n estudiantes con m notas, luego de mostrar el promedio, debe de decir si el estudiante aprobó o no y entregar la lista de los estudiantes aprobados.

Diseño:

El diseño del programa se basa en un ciclo que procesa estudiante por estudiante: primero se pide la cantidad de estudiantes ny la cantidad de notas por estudiante m, luego se recorre un *for* que se repite $n$veces. En cada repetición se recibe una línea con el nombre y sus $m$notas separadas por espacios (usando *split()*), se valida que la entrada tenga exactamente $m + 1$datos y, si no, se vuelve a pedir. Después se acumulan las notas en una variable suma, se calcula el promedio dividiendo entre my se compara con 3.0 para decidir si el estudiante aprueba; si aprueba, su nombre se agrega a la lista aprobados. Durante el proceso se imprime el promedio y el estado de cada estudiante, y al final se muestra la lista completa de los estudiantes aprobados.

Requerimientos:

- **RF01:** El programa debe pedir la cantidad de estudiantes n.
- **RF02:** El programa debe pedir la cantidad de notas por estudiante m.
- **RF03:** El programa debe permitir ingresar, para cada estudiante, el nombre y sus $m$notas separados por espacios.
- **RF04:** El programa debe validar que se ingresen exactamente $m + 1$datos por estudiante (nombre + $m$notas).
- **RF05:** El programa debe calcular el promedio de cada estudiante.
- **RF06:** El programa debe indicar si el estudiante aprueba o no con la regla: promedio ≥ 3.0.
- **RF07:** El programa debe guardar los nombres de los estudiantes aprobados y mostrarlos al final.

Historial de usuarios:

HU01: Ingresar la cantidad de estudiantes n.

HU02: Ingresar la cantidad de notas por estudiante m.

HU03: Leer los datos de cada estudiante en una línea: **nombre** + m**notas** separadas por espacios.

HU04: Verificar que se ingresaron exactamente $m + 1$datos; si no, pedir los datos otra vez.

HU05: Sumar las $m$notas del estudiante.

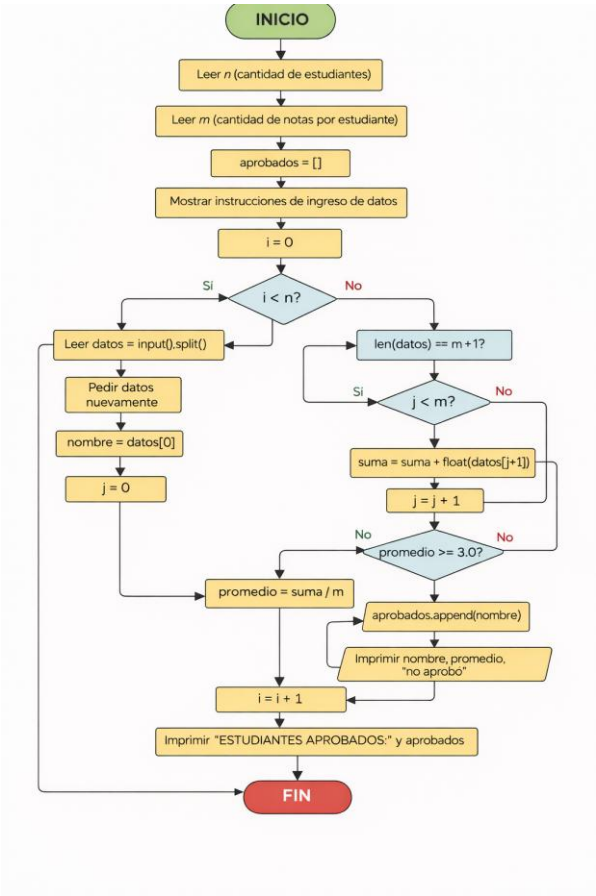
HU06: Calcular el promedio: promedio $= \frac{\text{suma}}{m}$.

HU07: Si el promedio es ≥ 3.0, mostrar “aprobó” y guardar el nombre en aprobados.

HU08: Si el promedio es <3.0, mostrar “no aprobó”.

HU09: Mostrar la lista final de “ESTUDIANTES APROBADOS”.

Diagrama de flujo:



Análisis de complejidad:

No se evidencian estructuras complejas en la resolución del problema, se puede notar que solo hay un ciclo for anidado junto al ciclo while, esto respecto al ciclo for principal, el análisis del costo y tiempo de este código se realiza a continuación en la siguiente tabla:

Tabla de Costo y Tiempo

FUNCION	COSTO	TIEMPO
n = int(input("Digite la cantidad de estudiantes: "))	C1	1
m= int(input("Digite la cantidad de notas que desea calcular por estudiante: "))	C2	1
aprobados = []	C3	1
print("Digite por cada estudiante: nombre y luego sus notas, \n si es decimal use punto (todo separado por espacios) y de enter entre cada estudiante: ")	C4	1
for i in range(n):	C5	n
datos = input("ingrese los datos: ").split()	C6	$n-1$
while len(datos)!= m+1:	C7	$(n-1)\sum_{i=1}^{n} t_i$
datos=input("Hubo un error, digite nuevamente nombre y notas separado por espacio")	C8	$(n-1)(\sum_{i=1}^{n} t_i$ -1)
nombre = datos[0]	C9	$n-1$
suma = 0.0	C10	$n-1$
for j in range(m):	C11	$(n-1)\sum_{j}^{m}$ tj
suma = suma +float(datos[j+1])	C12	$(n-1)(\sum_{j}^{m}$ tj-1)
promedio = suma / m	C13	$n-1$
if promedio >= 3.0:	C14	$n-1$
aprobados.append(nombre)	C15	$n-1$
print(nombre, "promedio:" , promedio, "aprobó ☺ ")	C16	$n-1$
else:	C17	$n-1$
print(nombre, "Promedio:",promedio,"no aprobó :c")	C18	$n-1$
print("ESTUDIANTES APROBADOS:", aprobados)	C19	1

$$\mathrm{T(n)} = \mathrm{c1} + \mathrm{c2} + \mathrm{c3} + \mathrm{c4} + \mathrm{c5}*\mathrm{n} + \mathrm{c6}*(\mathrm{n\text{-}1}) + \mathrm{c7}*(n-1)\sum_{i=1}^{n} t_i + \mathrm{c8}*(n-1)(\sum_{i=1}^{n} t_i \text{-}1) + \mathrm{c9}*n$$
$$- 1 + \mathrm{c10}*n - 1 + \mathrm{c11}*n - 1)\sum_{\mathrm{j}}^{\mathrm{m}} \mathrm{tj} + \mathrm{c12}*(n-1)(\sum_{j}^{m} tj - 1) + \mathrm{c13}*n - 1$$
$$+ \mathrm{c14}*n - 1 + \mathrm{c15}*n - 1 + \mathrm{c16}*n - 1 + \mathrm{c17}*n - 1 + \mathrm{c18}*n - 1 + \mathrm{c19}$$

Mejor y peor caso:

Mejor caso= Ti=1 y Tj=1

$$t_i = 1 \Rightarrow \sum_{i=1}^{n} t_i = \sum_{i=1}^{n} 1 = n$$

$$\boldsymbol{\sum_{i=1}^{n} t_i - 1 = n - 1}$$

$$t_j = 1 \Rightarrow \sum_{j=1}^{m} t_j = \sum_{j=1}^{m} 1 = m$$

$$\sum_{j=1}^{m} (t_j - 1) = \sum_{j=1}^{m} (1 - 1) = 0$$

$$T_{\text{mejor}}(n) = c_1 + c_2 + c_3 + c_4 + c_5 n + c_6(n-1) + c_7(n-1)\,n + c_8(n-1)(n-1) + c_9(n-1)$$
$$+ c_{10}(n-1) + c_{11}(n-1)\,m + c_{12}(n-1)\cdot 0 + c_{13}(n-1) + c_{14}(n-1)$$
$$+ c_{15}(n-1) + c_{16}(n-1) + c_{17}(n-1) + c_{18}(n-1) + c_{19}$$

$$T_{\text{mejor}}(n) = c_1 + c_2 + c_3 + c_4 + c_{19} + c_5 n + c_7 n(n-1) + c_8(n-1)^2 + (n-1)\,(C + c_{11} m)$$

$$T_{\text{mejor}}(n) = O(n^2 + nm)$$

Peor caso Ti=i y Tj=j

$$t_i = i \Rightarrow \sum_{i=1}^{n} t_i = \sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

$$\sum_{i=1}^{n} t_i - 1 = \frac{n(n+1)}{2} - 1$$

$$t_j = j \Rightarrow \sum_{j=1}^{m} t_j = \sum_{j=1}^{m} j = \frac{m(m+1)}{2}$$

$$\sum_{j=1}^{m}(t_j - 1) = \sum_{j=1}^{m}(j-1) = \frac{m(m+1)}{2} - m = \frac{m(m-1)}{2}$$

$$T_{\text{peor}}(n) = c_1 + c_2 + c_3 + c_4 + c_5 n + c_6(n-1) + c_7(n-1)\frac{n(n+1)}{2} + c_8(n-1)\left(\frac{n(n+1)}{2} - 1\right)$$

$$+c_9(n-1) + c_{10}(n-1) + c_{11}(n-1)\frac{m(m+1)}{2} + c_{12}(n-1)\frac{m(m-1)}{2} + c_{13}(n-1) + c_{14}(n-1) + c_{15}(n-1) + c_{16}(n-1) + c_{17}(n-1) + c_{18}(n-1) + c_{19}$$

$$T_{\text{peor}}(n) = c_1 + c_2 + c_3 + c_4 + c_{19} + c_5 n + (n-1)\left(C + c_7\frac{n(n+1)}{2} + c_8\left(\frac{n(n+1)}{2} - 1\right) + c_{11}\frac{m(m+1)}{2} + c_{12}\frac{m(m-1)}{2}\right)$$

$$T_{\text{peor}}(n) = O(n^3 + nm^2)$$